

Create a generative AI app that uses tools

In this exercise, you'll use the Microsoft Foundry portal and the Responses API to build an AI chat application. Then you'll integrate knowledge into your application by using the *web_search* and *file_search* tools.

This exercise takes approximately **30** minutes.

! **Note:** Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- An active [Azure subscription](#)
- [Visual Studio Code](#) installed
- [Python version 3.13.xx](#) installed*
- [Git](#) installed and configured
- [Azure CLI](#) installed

! * Python 3.14 is available, but some dependencies are not yet compiled for that release. The lab has been successfully tested with Python 3.13.12.

Create a Microsoft Foundry project

Microsoft Foundry uses projects to organize models, resources, data, and other assets used to develop an AI solution.

1. In a web browser, open the [Microsoft Foundry portal](#) at `https://ai.azure.com` to start building; signing in using your Azure credentials. Close any tips or quick start panes that are opened the first time you sign in.
2. If it is not already enabled, in the tool bar at the top of the page, enable the **New Foundry** option. Then, if prompted, create a new project with a unique name; expanding the **Advanced options** area to specify the following settings for your project:
 - **Foundry resource:** Use the default name for your resource (usually {project_name}-resource)
 - **Subscription:** Your Azure subscription
 - **Resource group:** Create or select a resource group
 - **Region:** Select any of the **AI Foundry recommended** regions in [this list](#)
3. Wait for your project to be created. Then view its home page.

Deploy a model

Next, let's deploy a model that you'll use in your chat application.

1. Now you're ready to explore models. On the **Discover** page, select the **Models** tab to view the Microsoft Foundry model catalog.
2. In the model catalog, search for `gpt-5.2`.
3. Review the model card, and then deploy it using the default settings.
4. When the model has been deployed, it will open in the model playground.

Experiment with tools in the playground

Before developing a chat application, let’s explore how the model responds in the playground. This will help you understand why grounding data matters.

1. After deploying your model, you should be in the playground with that model selected. If not, select **Build** in the top menu bar, then select **Deployments** on the left, and then select the model you deployed.
2. In the model playground, in the pane on the left, ensure that your **gpt-5.2** model is selected.
3. In the **Instructions** field, enter the following prompt:

CodeCopy

You are a travel assistant that provides information on travel services available from Margie's Travel.

4. In the chat pane, enter the query

What are some recommended tourist activities in New York next month?

 and review the response.

The response should be fairly generic — the model provides general knowledge based on its training data, but doesn’t have access to current information about what’s happening in New York next month.

5. In the pane on the left, under the instructions, in the **Tools** section, select **Add** and add the **web_search** tool.
6. In the chat pane, enter the same query

What are some recommended tourist activities in New York next month?

 and review the response.

This time, the model uses the *web_search* tool to find current information about activities in New York.

Create an app that uses tools

Now that you’ve seen how tools can extend a model’s capabilities in the playground, let’s build a client application that uses tools to provide travel advice for Margie’s Travel customers.

Get the endpoint

You’ll need an endpoint to connect to the model from a client application. In this exercise, we’re going to use the OpenAI SDK to chat with the model; and we’ll use the Azure OpenAI endpoint with Entra ID authentication to connect to it.

Note: As an alternative to Entra ID authentication, you could use the API Key for the project. using Entra ID authentication is preferred whenever possible.

1. On the menu bar, select the **Home** page.
2. Note the **Azure OpenAI Endpoint** displayed there.

Tip: You’ll use the **Azure OpenAI Endpoint** in this exercise, not the project endpoint!

Get the application files from GitHub

The initial application files you’ll need to develop your chat application are provided in a GitHub repo.

1. Open Visual Studio Code.

2. Open the command palette (*Ctrl+Shift+P*) and use the `Git:clone` command to clone the `https://github.com/microsoftlearning/mslearn-ai-studio` repo to a local folder (it doesn't matter which one). Then open it.

You may be prompted to confirm you trust the authors.

Prepare the application configuration

1. In Visual Studio Code, view the **Extensions** pane; and if it is not already installed, install the **Python** extension.
2. In the **Command Palette**, use the command `python:select interpreter`. Then select an existing environment if you have one, or create a new **Venv** environment based on your Python 3.1x installation.

 **Tip:** If you are prompted to install dependencies, you can install the ones in the *requirements.txt* file in the */labfiles/tools/python/tools-app* folder; but it's OK if you don't - we'll install them later!

3. In the Explorer pane, navigate to the folder containing the application code files at **/labfiles/tools/python/tools-app**. The application files include:
- **brochures** (a folder containing Margie's Travel brochures)
 - **.env** (the application configuration file)
 - **requirements.txt** (the Python package dependencies that need to be installed)
 - **tools-app.py** (the code file for the application)
4. In the **Explorer** pane, right-click the **tools-app** folder containing the application files, and select **Open in integrated terminal** (or open a terminal in the **Terminal** menu and navigate to the */labfiles/tools/python/tools-app* folder.)

 **Note:** Opening the terminal in Visual Studio Code will automatically activate the Python environment. You may need to enable running scripts on your system.

5. Ensure that the terminal is open in the **/labfiles/tools/python/tools-app** folder with the prefix **(.venv)** to indicate that the Python environment you created is active.
6. Install the OpenAI SDK, Azure identity, and other required packages by running the following command:

Code

Copy

```
pip install -r requirements.txt
```

7. In the **Explorer** pane, in the **/labfiles/tools/python/tools-app** folder, select the **.env** file to open it. Then update the configuration values to include the **Azure OpenAI Endpoint** and the name assigned to the deployment for the **gpt-5.2** model.

 **Tip:** Copy the **Azure OpenAI Endpoint** (not the project endpoint!) from the project home page in the Foundry portal, and enter the exact deployment name assigned to your deployment in the `MODEL_DEPLOYMENT` setting.

Save the modified configuration file.

Write code to implement chat with tools

1. In the **Explorer** pane, in the **/labfiles/tools/python/tools-app** folder, select the **tools-app.py** file to open it.
2. Review the existing code. You will add code to use the OpenAI SDK to access your model.

Tip: As you add code to the code file, be sure to maintain the correct indentation.

3. At the top of the code file, under the existing namespace references, find the comment **Import namespaces** and add the following code to import the namespace you will need to use the OpenAI SDK:

CodeCopy

```
# import namespaces
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider
```

4. In the **main** function, note that code to load the endpoint and key from the configuration file has already been provided. Then find the comment **Initialize the OpenAI client**, and add the following code to create a client for the OpenAI API:

CodeCopy

```
# Initialize the OpenAI client
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

openai_client = OpenAI(
    base_url=azure_openai_endpoint,
    api_key=token_provider
)
```

5. In the **main** function, find the comment **Create vector store and upload files**, and add the following code:

CodeCopy

```
# Create vector store and upload files
print("Creating vector store and uploading files...")
vector_store = openai_client.vector_stores.create(
    name="travel-brochures"
)

file_streams = [open(f, "rb") for f in glob.glob("brochures/*.pdf")]
if not file_streams:
    print("No PDF files found in the brochures folder!")
    return

file_batch = openai_client.vector_stores.file_batches.upload_and_poll(
    vector_store_id=vector_store.id,
    files=file_streams
)

for f in file_streams:
    f.close()

print(f"Vector store created with {file_batch.file_counts.completed} files.")
```

This code creates a vector store for your model, and uploads the brochures to it. We'll use this vector store with the *file_search* tool.

6. In the **main** function, note that code to request a user prompt until the user quits the app has been provided. Within this loop, find the **Get a response using tools** comment, and add the following code:

CodeCopy

```
# Get a response using tools
response = openai_client.responses.create(
    model=model_deployment,
    instructions="""
    You are a travel assistant that provides information on travel services available from
    Margie's Travel.

    Answer questions about services offered by Margie's Travel using the provided travel
    brochures.

    Search the web for general information about destinations or current travel advice.
    """,
    input=input_text,
    previous_response_id=last_response_id,
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store.id]
        },
        {
            "type": "web_search"
        }
    ]
)
print(response.output_text)
last_response_id = response.id
```

This code submits a prompt and specifies that the *file_search* tool can be used to search the vector store and the *web_search* tool can be used or general web searches.

7. Save the changes to the code file. Then, in the terminal pane, use the following command to sign into Azure.

CodeCopy

```
az login
```

!

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *-tenant* parameter. See [Sign into Azure interactively using the Azure CLI](#) for details.

8. When prompted, follow the instructions to sign into Azure. Then complete the sign in process in the command line, viewing (and confirming if necessary) the details of the subscription containing your Foundry resource.

9. After you have signed in, enter the following command to run the application:

CodeCopy

```
python tools-app.py
```

The program should run in the terminal (if not, resolve any errors and try again).

10. When prompted, enter `What's happening in San Francisco next month?` and review the response from your generative AI model.

The response should include information retrieved using the *web_search* tool.

11. Try this follow-up question: `What hotels does Margie's Travel offer there?`

The response should include information retrieved using the *file_search* tool.

12. When you're finished, enter `quit` to exit the program.

Clean up

If you've finished exploring Microsoft Foundry, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

1. Open the [Azure portal](#) and view the contents of the resource group where you deployed the resources used in this exercise.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.

[Prerequisites](#)

[Deploy a model](#)

[Experiment with tools in the playground](#)

Create an app that uses tools

[Clean up](#)